IoT Smart Industry Platform

FastAPI + ThingsBoard, with a flexible hierarchy and a dynamic frontend

---

Consolidated architecture report: objective, flexible entity hierarchy, ThingsBoard inheritance breakdown, alarm rules and white-labeling, frontend navigation, and the development checklist.

1. Project objective

The core objective is to use **ThingsBoard as the underlying IoT data engine**, build a **solid, scalable API** on top of it, and, most importantly, a **frontend far more capable and flexible than ThingsBoard's native one**: both the administrator and the end user can create their own dashboards and solutions, select any sensor and telemetry key, see its full history, generate aggregates (average, maximum, minimum) over any time range, and choose how to visualize the data.

This project is explicitly framed as a **proof of concept**: its purpose is to improve on ThingsBoard's frontend, to practice building a real FastAPI service end to end, and, together with former coworkers, to explore whether something better and more scalable can be built. Even so, the intent is to **build it properly from day one**. The backend stays close to ThingsBoard's own capabilities rather than introducing a new event-driven layer - the focus of this design is the entity model, the flexible hierarchy, and the frontend.

Initial data source

ThingsBoard itself supports **emulating devices and assets** through its own device profiles, without any physical hardware. This emulated data is the initial data source, and the same integration can later point at real sensors without architectural changes.

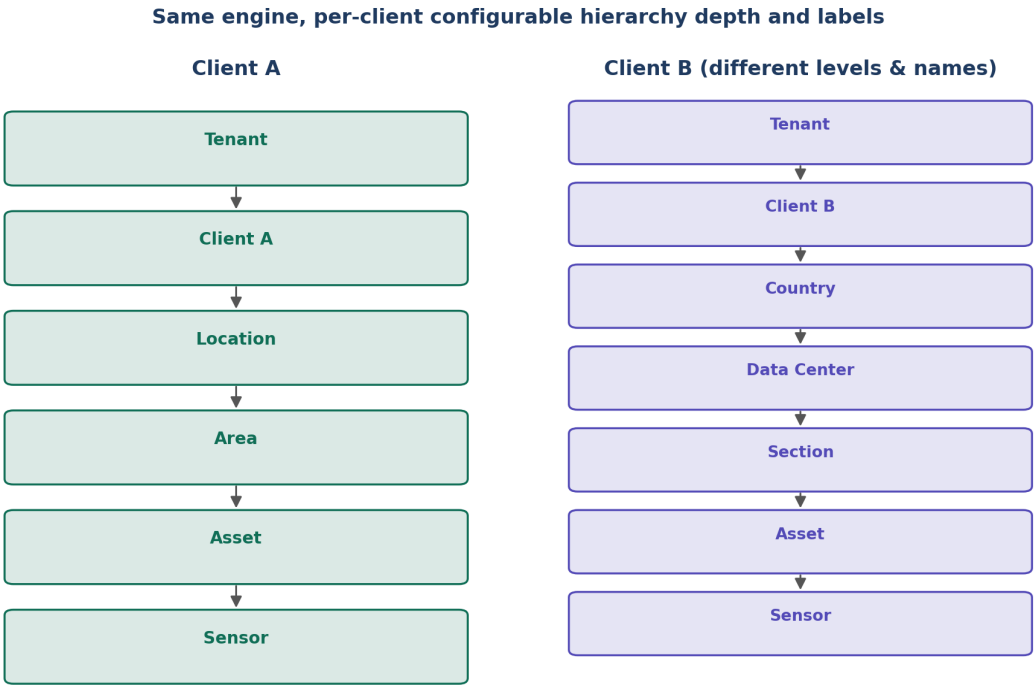
Project stages

- **Stage 1:** a scalable FastAPI service that authenticates against ThingsBoard and exposes devices, assets, telemetry, and attributes.
- **Stage 2:** a dynamic frontend consuming this API, with per-sensor telemetry exploration, aggregation, and fully user/admin-configurable dashboards.

2. Flexible entity hierarchy

The baseline hierarchy is **Tenant** → **Client** → **Location** → **Area** → **Asset** → **Sensor/Gateway**, with names distinct from ThingsBoard's own to differentiate the product. Clients can be nested recursively (e.g. IAC as a parent with Clarios and Lhoist as sub-clients).

Key requirement: the hierarchy must be configurable per Client. Some Clients won't use intermediate levels at all; others will want different labels and a different number of levels (e.g. Country / Data Center / Section instead of Location / Area).



How this is solved

- A **single generic relation type** (e.g. "Contains") is used for every parent-child link in the tree, regardless of what the level represents - avoiding a combinatorial explosion of relation types (clientToLocation, locationToArea, countryToDataCenter, etc.).
- Each intermediate node is still a ThingsBoard **Asset**, tagged with a **hierarchyLevelId** attribute that points to a level definition.
- A custom table, **hierarchy_level_definitions** (id, tb_customer_id, level_order, label, icon), defines - per Client - which levels exist and what they are called. This is metadata ThingsBoard cannot express.
- Permission scope becomes generic too: roles point to any **tb_entity_id**, not to a fixed level name like "location" or "area".

As a rigid fixed-name chain, this would not scale past the first Client with a different structure. Generalized this way - generic Assets and Relations, plus a per-Client level definition table - it scales cleanly, and it is a natural refinement of the same idea rather than a redesign.

3. Telemetry and attributes per entity

Every entity carries two distinct kinds of data, both sourced from ThingsBoard: **telemetry** (time-series values, e.g. temperature, pressure, flow rate) and **attributes** (single current values, e.g. latitude, longitude, thresholds, firmware version). Attributes can be created or edited directly from the frontend - for example, setting latitude/longitude via a map pin, or a threshold via a simple form.

Aspect	Telemetry	Attributes
Nature	Time-series data points over time	Single current value per key
Examples	temperature, pressure, flowRate, vibration, energy	latitude, longitude, threshold, install date, firmware version
Source	ThingsBoard	ThingsBoard, editable from the frontend
Typical UI	Charts, gauges, aggregated cards (avg/max/min)	Forms, map pin editor, threshold sliders

The Data Classifier

A backend component that suggests how to visualize each telemetry key (chart, card, gauge, badge), combining: the data type ThingsBoard already returns (numeric, boolean, string), the unit - which ThingsBoard does not provide natively, so it is stored in a custom **telemetry_key_catalog** table - and name/type heuristics as a fallback for keys not yet catalogued.

4. What inherits from ThingsBoard, entity by entity

The guiding rule: reuse what ThingsBoard already solves well via its API, and build a custom table only for what it cannot express. Nothing that already lives in ThingsBoard gets duplicated as a local table - that would create two sources of truth.

A - Fully native (read/write via the ThingsBoard API, no local table)

- ■ Tenant
- ■ Client (Customer)
- ■ Users + base Authority
- ■ Assets
- ■ Devices (Sensor/Gateway)
- ■ Attributes
- ■ Telemetry
- ■ Relations

B - Modeled inside ThingsBoard via convention (also no local table)

- ■ Location / Area / any intermediate level → Asset with a hierarchyLevelId attribute
- ■ Nested Clients → a native Relation between two Customers, not a parent_client_id column
- ■ Theme / White-label → SERVER_SCOPE attributes on the Customer entity (logoUrl, primaryColor, secondaryColor, layoutPrefs)

C - Hybrid: native base, thin custom complement

Entity	Explanation
AlarmRule	The condition lives in the native Rule Chain of the Device Profile; only the recipient emails need a local table: alarm_recipients (id, tb_rule_node_id, email).
Role / Permission	Base Authority is native; granular permissions per arbitrary hierarchy node require a local table: roles (id, name, tb_entity_id).

D - Fully custom (the platform's real differentiator)

`hierarchy_level_definitions, telemetry_key_catalog, dashboard_configs, favorites, onboarding_flows`

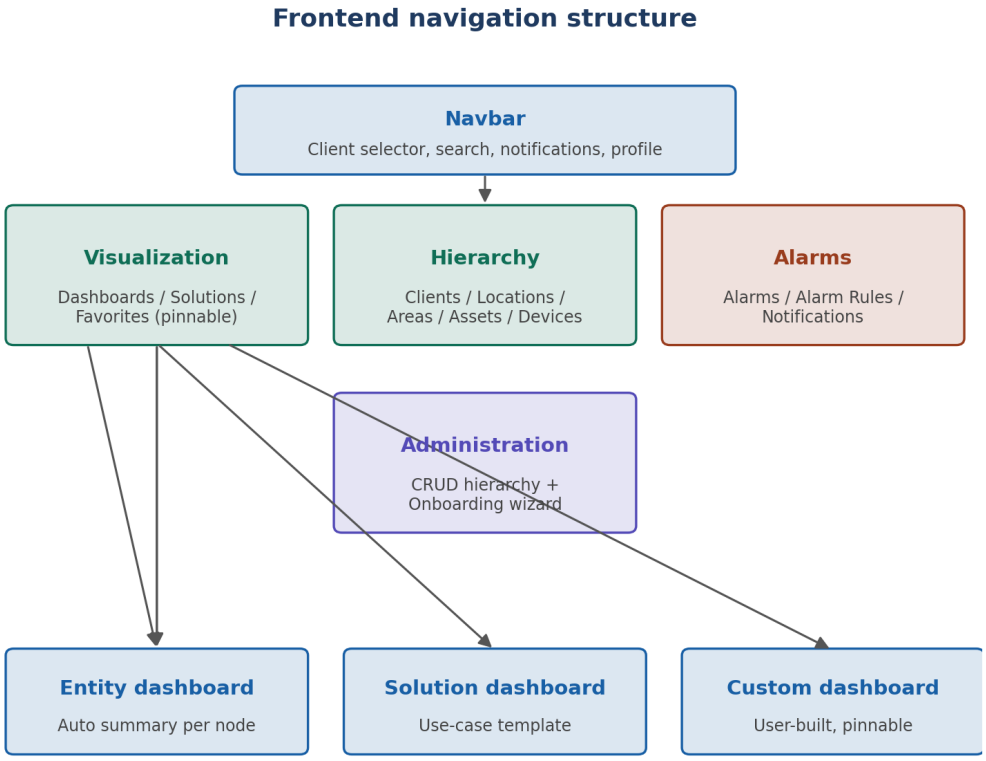
5. Alarm rules and white-labeling

Each Client can edit the alarm rules of its own Device Profiles directly - the "Create/Clear Alarm" nodes of the native Rule Chain - and configure which email addresses receive each alarm, rather than a single fixed recipient per tenant.

White-labeling is scoped to **logo, color palette, and layout** per Client. A custom domain/subdomain and custom login-page branding were explicitly considered and dropped from scope.

- ■ Logo and favicon upload per Client
- ■ Custom color palette (primary, secondary, accent, background) per Client
- ■ Custom layout preferences (sidebar position, density, card vs. table default views)
- ■ Fallback to a default theme when a Client has not configured white-labeling

6. Frontend navigation



The **Navbar** holds the Client selector, global search, notifications, and profile. The **Leftbar is collapsible**, with a dynamic active section based on the entity currently selected in the hierarchy.

Leftbar tab groups

- **Visualization:** Dashboards, Solutions, Favorites (with a pinnable default dashboard).
- **Hierarchy:** Clients, Assets, Devices - Locations/Areas are browsed within the same tree.
- **Alarms:** Alarms, Alarm Rules, Notifications.
- **Administration:** full CRUD of the hierarchy plus **onboarding flows** - a wizard to set up a new Client with its complete structure in one guided flow, similar to a bulk hierarchy wizard.

Three dashboard types

Type	Description
Entity dashboard	Automatic summary shown when selecting any node of the hierarchy (Location, Area, Asset, or Sensor).
Solution dashboard	Pre-built template for a specific use case, offered by the administrator or a catalog.

Type	Description
Custom dashboard	Built by the end user, with fixed or dynamic sensors/telemetry, pinnable as the default view.

7. General architecture

The frontend requests dynamic data from the FastAPI service, which authenticates against and queries ThingsBoard (devices, assets, telemetry, attributes). The Data Classifier determines how each telemetry key should be visualized, and the API sends email notifications when an alarm fires. The backend intentionally stays close to what ThingsBoard already provides rather than introducing new infrastructure.

Main components

- **Frontend:** dynamic UI where both admins and end users build dashboards and solutions; selects any sensor and telemetry key to view raw data or aggregates as charts or tables.
- **FastAPI service:** business logic and authentication layer; exposes its own endpoints rather than ThingsBoard's raw shape.
- **ThingsBoard:** IoT data engine (devices, assets, telemetry, attributes, hierarchy, native Relations, native Authority, native Rule Engine for alarms).
- **Data classifier:** infers the right visualization based on key, unit, and data type.
- **Notifications:** email delivery for alarms.

Documentation and deployment

- Versioned Bruno collection documenting the business-logic endpoints.
- docker-compose with the API and database.

8. ThingsBoard limitations and how this project addresses them

ThingsBoard limitation	Proposed solution
Dashboards are built widget by widget; only admins/tenants can meaningfully configure them.	Scalable frontend where both admins and end users create their own dashboards and solutions.
Dynamic aliases resolve the data source, not the choice of visualization.	Custom classifier that infers which visualization to show based on the actual telemetry.
Fixed hierarchy (tenant -> customer -> asset/device), no configurable intermediate levels.	Extended, per-Client configurable hierarchy (Location/Area or Country/Data Center/Section, etc.).
No native multi-brand support; a single tenant looks the same to every customer.	Per-client white-labeling: logo, colors, and layout.
No built-in way for a user to pick one sensor/key and see aggregates across charts/tables.	Dedicated sensor + telemetry key explorer with avg/max/min aggregation and chart/table views.

9. Final development checklist

Data model and hierarchy

- ■ Base hierarchy: Tenant → Client → Location → Area → Asset → Sensor/Gateway
- ■ Nestable/concatenable Clients (e.g. IAC → Clarios, Lhoist)
- ■ Per-Client configurable hierarchy depth and labels (hierarchy_level_definitions)
- ■ Single generic relation type ("Contains") for all parent-child links
- ■ Every entity exposes both telemetry and attributes, both sourced from ThingsBoard
- ■ Attributes editable from the frontend (e.g. lat/long via map, thresholds via forms)
- ■ No duplication of ThingsBoard-native entities as local tables

Backend (Stage 1 — API)

- ■ Scalable FastAPI service authenticated against ThingsBoard
- ■ Access to devices, assets, telemetry, and attributes via a custom API layer
- ■ Attribute CRUD (create/update/delete), including latitude/longitude and thresholds
- ■ Key classifier that infers the right visualization type
- ■ Aggregation endpoints (average, maximum, minimum) per sensor and telemetry key
- ■ Dashboard/solution builder usable by both admins and end users
- ■ Alarm rules editable per Client from its own Device Profiles
- ■ Configurable alarm email recipients per Client
- ■ Configurable theming per Client (colors and layout)
- ■ API documentation with Bruno
- ■ Fully dockerized (docker-compose)

Additional selected features

- ■ Granular roles and permissions per hierarchy node, built on ThingsBoard's native Authority
- ■ Historical telemetry export to CSV/Excel
- ■ Real multi-tenancy (logical instance per Tenant)
- ■ Favorites/pins for specific Sensors/Gateways or keys
- ■ Automated tests (key classifier unit tests + ThingsBoard mock integration tests)
- ■ "Presentation" mode (read-only dashboard)
- ■ AI integration (natural-language chat about sensor status)
- ■ Onboarding wizard for new Clients (bulk hierarchy setup)

Stage 2

- ■ Frontend dashboard/solution builder consuming the full API, with per-sensor telemetry exploration and aggregation

---

Document generated as a reference baseline to kick off development.